

10기_ML팀_권지원_요약본

Chapter 10. 차원 축소 (Dimensionality Reduction)

1. 차원 축소 개요

차원 축소란?

- 매우 많은 피처로 구성된 다차원 데이터 세트의 차원을 줄여 새로운 차원의 데이터 세트를 생성하는 기법임

고차원 데이터의 문제점 (차원의 저주)

- 데이터 포인트 간의 거리가 기하급수적으로 멀어지며, 희소(Sparse)한 데이터 구조를 가짐
- 적은 차원에서 학습된 모델에 비해 예측 신뢰도가 크게 떨어짐
- 피처 간 상관관계가 높아질 가능성이 증가하여 선형 모델에서 다중 공선성 문제가 발생함

차원 축소의 두 가지 방식

방식	설명	특징
피처 선택 (Feature Selection)	불필요한 피처를 제거하거나 중요한 피처만 선택함	원본 피처를 유지함
피처 추출 (Feature Extraction)	기존 피처를 단순 압축하는 것이 아니라, 피처를 더 잘 설명하는 새로운 공간으로 매핑함	새롭게 추출된 특성은 기존 피처와 완전히 다른 값이 됨

- 데이터 유형별 활용 영역

- 이미지 데이터: 잠재적 특성을 피처로 도출하여 이미지 변환 및 압축에 활용
- 텍스트 데이터: 단어 구성에 숨겨진 시맨틱(Semantic) 의미나 토픽(Topic)을 잠재 요소로 추출 (주로 SVD, NMF 알고리즘 기반)

- 대표 알고리즘: PCA, LDA, SVD, NMF

매니폴드 학습 (Manifold Learning)

- 매니폴드 가정: 대부분의 고차원 데이터는 저차원 매니폴드(Manifold)에 가깝게 놓여 있다는 가정에 기반함
- d차원 매니폴드: n 차원 공간의 일부로, d 차원 초평면처럼 보이는 구조 ($d < n$)
- 단순한 투영(Projection)이 아닌, 매니폴드를 '펼쳐서' 저차원으로 변환해야 원본의 데이터 구조를 보존할 수 있음 (예: 스위스롤 데이터셋)

2. PCA (Principal Component Analysis)

기본 아이디어

- 변수 간의 상관관계를 이용하여 주성분(Principal Component)을 추출해 차원을 축소함
- 정보 유실 최소화가 주된 목표임
- 가장 높은 분산(=데이터의 변동성)을 가지는 데이터의 축을 PCA의 주성분으로 삼아 데이터를 투영함

주성분(PC) 도출 방식

- 첫 번째 주성분 (PC1): 데이터가 가장 많이 퍼진 방향 (최대 분산 축)
- 두 번째 주성분 (PC2): PC1과 직교하면서 그 다음으로 분산이 큰 방향
- $n + 1$ 번째 주성분 (PC $n+1$): PC n 과 직교하면서 그 다음으로 분산이 큰 방향
- 결과적으로 "벡터 축의 개수 = 차원의 수"가 되며, 작은 수의 주성분만으로 원본 데이터의 총 변동성을 대부분 설명할 수 있음

선형대수 관점에서의 PCA

- 핵심: 공분산 행렬을 고유값 분해(Eigendecomposition)하여 얻은 고유벡터를 새로운 축으로 사용함
- 공분산 행렬 (Σ): 모든 피처 간의 관계(분산+공분산)를 담은 전체 정보
- 고유값 분해 수식:

$$\Sigma = Q \Lambda Q^T$$

구성요소	설명	특징
Q	고유벡터 행렬 (새로운 좌표축)	직교 행렬 (대칭 행렬이므로 항상 직교 고유벡터가 보장됨)
Λ	고유값을 모은 대각행렬	각 축의 분산 크기를 의미하며, 큰 순서대로 정렬됨
고유벡터	PCA의 주성분 벡터	입력 데이터의 분산이 가장 큰 방향을 나타냄
고유값	고유벡터의 크기	해당 방향(고유벡터)에서의 분산 크기를 의미함

PCA 수행 스텝

1. 입력 데이터 세트의 공분산 행렬 생성
2. 공분산 행렬의 고유벡터와 고유값 계산
3. 고유값이 가장 큰 순으로 K 개(PCA 변환 차수)의 고유벡터 추출
4. 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환

사이킷런(Scikit-learn) 구현 포인트

- 주의사항: PCA 적용 전 반드시 `StandardScaler` 를 통한 데이터 스케일링이 필요함 (스케일이 다를 경우 분산이 큰 특성이 주성분을 독점할 우려가 있음)

Python

```
# PCA 변환 기본 흐름
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. 스케일링
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2. PCA 적용 (K 차원으로 축소)
pca = PCA(n_components=K)
X_pca = pca.fit_transform(X_scaled)

# 3. 분산 설명 비율 확인
print(pca.explained_variance_ratio_)
```

적절한 차원 수(K) 선택 방법

- 방법 1: 누적 분산 비율이 95%가 되는 최소 차원 수 지정 (파라미터: `n_components=0.95`)
- 방법 2: '설명된 분산 비율 vs 차원 수' 그래프를 그려 변곡점(Elbow point) 확인

3. LDA (Linear Discriminant Analysis)

기본 아이디어

- PCA와 유사하게 저차원 공간에 투영하여 차원을 축소하지만, 접근 목적이 다름

구분	학습 방식	목적 및 핵심 원리
PCA	비지도학습	입력 데이터의 분산이 가장 큰 축을 찾음
LDA	지도학습	클래스 결정 값을 사용하여, 클래스를 최대한 분리할 수 있는 축을 찾음

핵심 원리

- 클래스 간 분산(Between-class scatter)을 최대화하고, 클래스 내부 분산(Within-class scatter)을 최소화 하는 방향으로 축을 도출함
- 즉, 클래스 분리를 최대화하는 방향이 LDA의 주성분이 됨

LDA 수행 스텝

1. 클래스 내부 분산 행렬(S_W)과 클래스 간 분산 행렬(S_B) 계산 (입력 데이터의 결정 값 클래스별 평균 벡터 기반)
2. $S_W^{-1} S_B$ 를 고유값 분해
3. 고유값이 가장 큰 순으로 K 개의 고유벡터 추출
4. 추출된 고유벡터로 입력 데이터를 새롭게 변환

사이킷런(Scikit-learn) 구현 포인트

- 비지도학습인 PCA와 달리, 지도학습이므로 `fit` 단계에서 클래스 레이블(`yy`) 값이 반드시 필요함

Python

```
# LDA 변환 기본 흐름
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

lda = LinearDiscriminantAnalysis(n_components=K)
lda.fit(X_scaled, y) # y(클래스 레이블) 필수
X_lda = lda.transform(X_scaled)
```

4. SVD (Singular Value Decomposition)

기본 아이디어

- 원본 행렬을 3개의 행렬로 분해하는 기법
- 수식:

$$A = U \Sigma V^T$$

- **특이값(Singular Value):** 행렬 Σ 내의 0이 아닌 대각원소들로, 데이터의 중요도를 나타내는 지표임 (값이 클수록 중요함)
- 정방행렬에만 적용 가능한 PCA와 달리, 행과 열의 크기가 다른 행렬에도 적용 가능함

행렬 분해 구성요소

구성요소	설명	특징
U	행 방향 특징을 담은 행렬	m $m \times m$ 크기의 직교행렬
Σ	중요도 정보를 담은 대각행렬	m $m \times n$ 크기 (특이값이 대각원소임)
V^T	열 방향 특징을 담은 행렬	n $n \times n$ 크기의 직교행렬

- U 와 V 에 속한 벡터는 특이 벡터(Singular Vector)라 부르며, 서로 직교하는 성질을 가짐

Truncated SVD

- Σ 의 대각원소 중 상위 P 개만 추출하고, 이에 대응하는 U, V 의 원소도 함께 제거하여 차원을 더욱 줄인 형태임
- 원본 행렬로 완벽하게 복원되지는 않지만, 상당한 수준으로 원본 행렬 근사가 가능하여 데이터 압축 및 차원 축소에 활발히 사용됨

사이킷런(Scikit-learn) 구현 포인트 및 특징

- `Scipy`의 `svds` 모듈과 달리, 사이킷런의 `TruncatedSVD` 클래스는 U, Σ, V^T 행렬을 직접 반환하지 않음
- PCA와 동일하게 `fit()`, `transform()` 메서드를 통해 저차원 변환된 결과 행렬만 반환함

Python

```
# Truncated SVD 변환 기본 흐름
from sklearn.decomposition import TruncatedSVD

tsvd = TruncatedSVD(n_components=K)
X_tsvd = tsvd.fit_transform(X)
```

PCA vs SVD 비교

- **공통점:** 데이터를 스케일링(중심화)한 뒤 SVD를 적용하면 PCA와 완전히 동일한 변환 결과를 얻음 (사이킷런의 PCA 클래스는 내부적으로 SVD 알고리즘을 사용해 구현됨)
- **차이점:** PCA는 밀집 행렬(Dense Matrix)에만 적용할 수 있으나, SVD는 희소 행렬(Sparse Matrix)에도 적용이 가능함

5. NMF (Non-negative Matrix Factorization)

기본 아이디어

- 원본 행렬을 음수를 포함하지 않는 두 개의 양수 행렬로 분해하는 기법
- 수식:

$$V \approx W \times H$$

분해 행렬	크기	의미 및 활용
W	$m \times p$	각 데이터가 잠재 특징과 얼마나 관련되어 있는지 나타냄 (특징 추출 및 차원 축소에 활용)
H	$p \times n$	잠재 특징이 어떤 원본 요소들로 구성되어 있는지 나타냄 (토픽 모델링 등에 활용)

- p : 도출하고자 하는 잠재 요소의 수 (축소할 차원의 수)

증배 갱신 규칙 (Multiplicative Update Rules)

- W 와 H 에 초기값을 무작위로 설정한 후, 원본 행렬 V 와 분해 행렬 곱 WH 간의 거리를 최소화하도록 값을 반복적으로 업데이트함
- **곱셈 방식을 사용하는 이유**: NMF는 오직 '양수'로만 연산을 수행해야 하므로, 더하기/빼기 방식을 사용하면 연산 과정에서 음수가 발생할 수 있기 때문임
- 원본 행렬 V 와 추정 행렬 WH 의 차이가 거의 0이 될 때까지 해당 과정을 반복함

특징 및 활용

- SVD 분해와 달리, 분해된 행렬(W, H)의 값이 모두 양수이기 때문에 결과에 대한 직관적인 해석이 용이함
- 주로 텍스트 데이터의 **토픽 모델링(Topic Modeling)**, 영상/이미지 데이터의 **특징 추출** 등에 광범위하게 활용됨

사이킷런(Scikit-learn) 구현 포인트

Python

```
# NMF 변환 기본 흐름
from sklearn.decomposition import NMF

nmf = NMF(n_components=K)
X_nmf = nmf.fit_transform(X) # 원본 데이터 행렬(X)의 모든 값은 양수여야 함
```